BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG CƠ SỞ HCM
KHOA CÔNG NGHỆ THÔNG TIN 2

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

ĐỀ TÀI SỐ 09: HỆ THỐNG GIÁM SÁT AO NUÔI TÔM IOT (SMART AQUACULTURE MONITOR)



Nhóm sinh viên thực hiện: Nhóm 06
Lớp chuyên ngành: D24CQAT01-N
Năm học: 2026 – 2027

TP HCM, Tháng 10 năm 2026

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ TỶ LỆ ĐÓNG GÓP

STT	Họ và tên Mã SV	Vai trò	Lớp / Chức năng phụ trách	ĐG %
1	Đào Thế Quân N24DCAT060	Nhóm trưởng / Tích hợp	MainWindow (extends BaseWindow), InvalidThresholdException, Main, thiết lập Git repo	10
2	Trần Nhật Tiến N24DCAT79	UI & Cấu hình	ThresholdDialog, AboutDialog, PropertiesHelper, bắt ngoại lệ với try-catch	10
3	Trần Nguyên Cầu N24DCAT010	Model cốt lõi	Sensor (abstract), TemperatureSensor, ném InvalidThresholdException trong setter	10
4	Lê Hồng Phát N24DCAT054	Model dữ liệu	PHSensor, DissolvedOxygenSensor, thuật toán nhiễu Gauss Random.nextGaussian()	10
5	Lê Hữu Phát N24DCAT055	Model & Data	SalinitySensor, Reading (immutable), thiết kế 12 kịch bản kiểm thử (Test Cases)	10
6	Nguyễn Hoàng Hưng N24DCAT031	Observer & Logic	Farm, Pond, Alert, thiết lập cấu trúc quan hệ Composition	10
7	Chiêm Lâm Khánh Tân N24DCAT070	Observer & Event	SensorHub, SensorListener (interface), kết nối lắng nghe và phân phát sự kiện	10
8	Trần Phú Cường N21DCAT008	UI & Graphics	GaugePanel (tự vẽ Graphics2D đổi màu Xanh/Vàng/Đỏ), DashboardPanel	10
9	Nguyễn Ngọc Quy N24DCAT064	UI & Background	LogTableModel (extends AbstractTableModel, giữ 200 dòng), điều khiển Timer 1s	10
10	Nguyễn Tấn Phát N24DCAT056	Chart & Bonus	Controllable (interface), Aerator (tự động bật tắt), tích hợp JFreeChart	10

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh bài toán

Nuôi tôm công nghệ cao đang là một trong những ngành kinh tế mũi nhọn. Tuy nhiên, tôm là loài cực kỳ nhạy cảm với biến động của môi trường nước. Các chỉ số như Nhiệt độ, Độ pH, Lượng Oxy hòa tan (DO) và Độ mặn nếu vượt khỏi ngưỡng an toàn trong thời gian ngắn có thể dẫn đến hiện tượng tôm chết hàng loạt. Việc giám sát thủ công truyền thống tốn nhiều nhân lực, chậm trễ trong việc phát hiện sự cố và thiếu tính đồng bộ.

Sự ra đời của công nghệ IoT (Internet of Things) cho phép thu thập dữ liệu liên tục 24/7. Nhằm mô phỏng và giải quyết bài toán thực tế này, nhóm quyết định xây dựng phần mềm "Hệ thống Giám sát Ao nuôi tôm IoT", ứng dụng triệt để các nguyên lý Lập trình hướng đối tượng (OOP) bằng ngôn ngữ Java để thiết kế một kiến trúc phần mềm linh hoạt, dễ mở rộng và trực quan.

1.2. Mục tiêu đề tài

Về mặt kỹ thuật: Xây dựng thành công hệ thống phần mềm Desktop bằng Java Swing, kết hợp thư viện đồ họa Graphics2D và JFreeChart. Áp dụng chuẩn xác 4 nguyên lý OOP (Trừu tượng, Đóng gói, Kế thừa, Đa hình) và các Design Pattern (Observer, Singleton).

Về mặt nghiệp vụ: Mô phỏng sinh động dữ liệu thực tế của các cảm biến môi trường (có tính đến nhiễu ngẫu nhiên), hiển thị cảnh báo theo thời gian thực và tự động hóa việc điều khiển thiết bị chấp hành khi có sự cố.

1.3. Phạm vi hệ thống

🎬 Mô phỏng dữ liệu cho một trang trại (Farm) gồm nhiều ao nuôi (Pond). Mỗi ao gắn 4 loại cảm biến cơ bản: Nhiệt độ, pH, DO, Độ mặn.

🎬 Hệ thống không kết nối phần cứng thật mà sử dụng Engine giả lập chạy ngầm (Swing Timer) để sinh dữ liệu mỗi 1 giây (phân phối chuẩn Gaussian).

🎬 Lịch sử giám sát được lưu trữ trong bộ nhớ đệm với giới hạn tối đa 200 dòng (sử dụng cấu trúc LinkedList) để tối ưu hiệu năng.

1.4. Danh sách chức năng hệ thống

STT	Tên chức năng	Mô tả chi tiết	Phân loại
1	Bảng điều khiển (Dashboard)	Hiển thị 4 đồng hồ đồ họa kim đo thời gian thực cho 4 chỉ số, tự động đổi màu Đỏ khi vượt ngưỡng an toàn.	Bắt buộc
2	Cấu hình ngưỡng an toàn	Cho phép người dùng tùy chỉnh ngưỡng Min/Max cho từng cảm biến. Lưu trữ cấu hình ra file app.properties.	Bắt buộc
3	Nhật ký hệ thống (Log)	Bảng JTable lưu 200 dòng lịch sử gần nhất, hiển thị thông báo "Bình thường" hoặc cảnh báo "Vượt ngưỡng".	Bắt buộc
4	Biểu đồ phân tích	Vẽ biểu đồ TimeSeries (Động học 60s) và BarChart (Thống kê số lần cảnh báo) bằng JFreeChart.	Bắt buộc
5	Bắt ngoại lệ nhập liệu	Tự động báo lỗi (chặn crash) khi cấu hình Min lớn hơn Max.	Bắt buộc
6	Tự động điều khiển thiết bị	Mô phỏng thiết bị Máy sục khí (Aerator) tự động bật khi hàm lượng Oxy hòa tan (DO) tụt xuống mức nguy hiểm.	BONUS

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu và các ca sử dụng

Smart Aquaculture Monitor là ứng dụng Desktop Java Swing mô phỏng việc thu thập các chỉ số môi trường ao nuôi tôm. Thiết kế hệ thống tập trung vào ba nhiệm vụ cốt lõi: sinh dữ liệu cảm biến, phân phối sự kiện và trình bày dữ liệu trực quan trên giao diện. Tác nhân chính (Actor) là người quản lý ao nuôi. Bộ định thời (Timer) của ứng dụng đóng vai trò tự động thực hiện lấy mẫu; trong khi người dùng thao tác trực tiếp với các chức năng giám sát, cấu hình và xem biểu đồ.

Ca sử dụng	Thao tác và kết quả	Phạm vi hiện tại
Giám sát ao nuôi	Mở ứng dụng, quan sát bốn đồng hồ và nhật ký.	Khởi tạo một ao P1; có danh sách chọn ao nhưng chưa có xử lý đổi ao.
Cấu hình ngưỡng	Nhập Min/Max, kiểm tra hợp lệ và lưu app.properties.	Hỗ trợ nhiệt độ và pH.
Xem biểu đồ	Mở cửa sổ có biểu đồ thời gian và số lần cảnh báo.	Nhận sự kiện từ lúc mở cửa sổ.
Xem thông tin	Chọn Help → About để mở AboutDialog.	Hộp thoại thông tin riêng.
Điều khiển sục khí	Phản ứng tự động theo dữ liệu DO.	Đổi trạng thái trong mô hình và ghi thông báo console.

Luồng giám sát bắt đầu khi MainWindow tạo Farm, Pond và bốn Sensor, sau đó khởi động SimulationEngine. Mỗi nhịp lấy mẫu, hệ thống tạo Reading và phát sự kiện. Nếu giá trị thấp hơn Min hoặc cao hơn Max, một Alert được phát bổ sung. Các đối tượng nhận sự kiện cập nhật phần hiển thị hoặc trạng thái tương ứng.

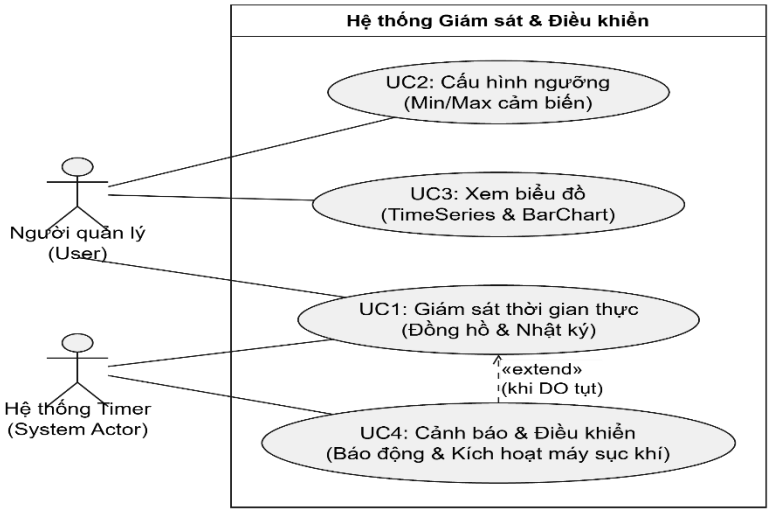
Phân tích yêu cầu và Sơ đồ Use Case

Hệ thống giám sát ao nuôi tôm IoT được thiết kế với hai tác nhân (Actor) chính:

1. **Người quản lý (User):** Tương tác trực tiếp với giao diện để theo dõi, cấu hình và xem thống kê.

2. **Hệ thống Timer (System Actor):** Luồng chạy ngầm, tự động lấy mẫu dữ liệu theo chu kỳ.

- **UC1 - Giám sát thời gian thực:** Xem 4 đồng hồ đồ họa và nhật ký cập nhật mỗi giây.
- **UC2 - Cấu hình ngưỡng:** Thay đổi và lưu trữ ngưỡng Min/Max cho từng cảm biến.
- **UC3 - Xem biểu đồ:** Mở cửa sổ phân tích đồ thị TimeSeries và BarChart.
- **UC4 - Cảnh báo & Điều khiển (Bonus):** Hệ thống tự động cảnh báo đỏ và kích hoạt máy sục khí khi Oxy hòa tan (DO) tụt giảm.



2.2. Cấu trúc package và trách nhiệm

Package gốc là vn.edu.ptit.oop.nhom06. SensorHub và SensorListener nằm trong model, không nằm trong một package observer riêng. Lớp Main là điểm vào của ứng dụng.

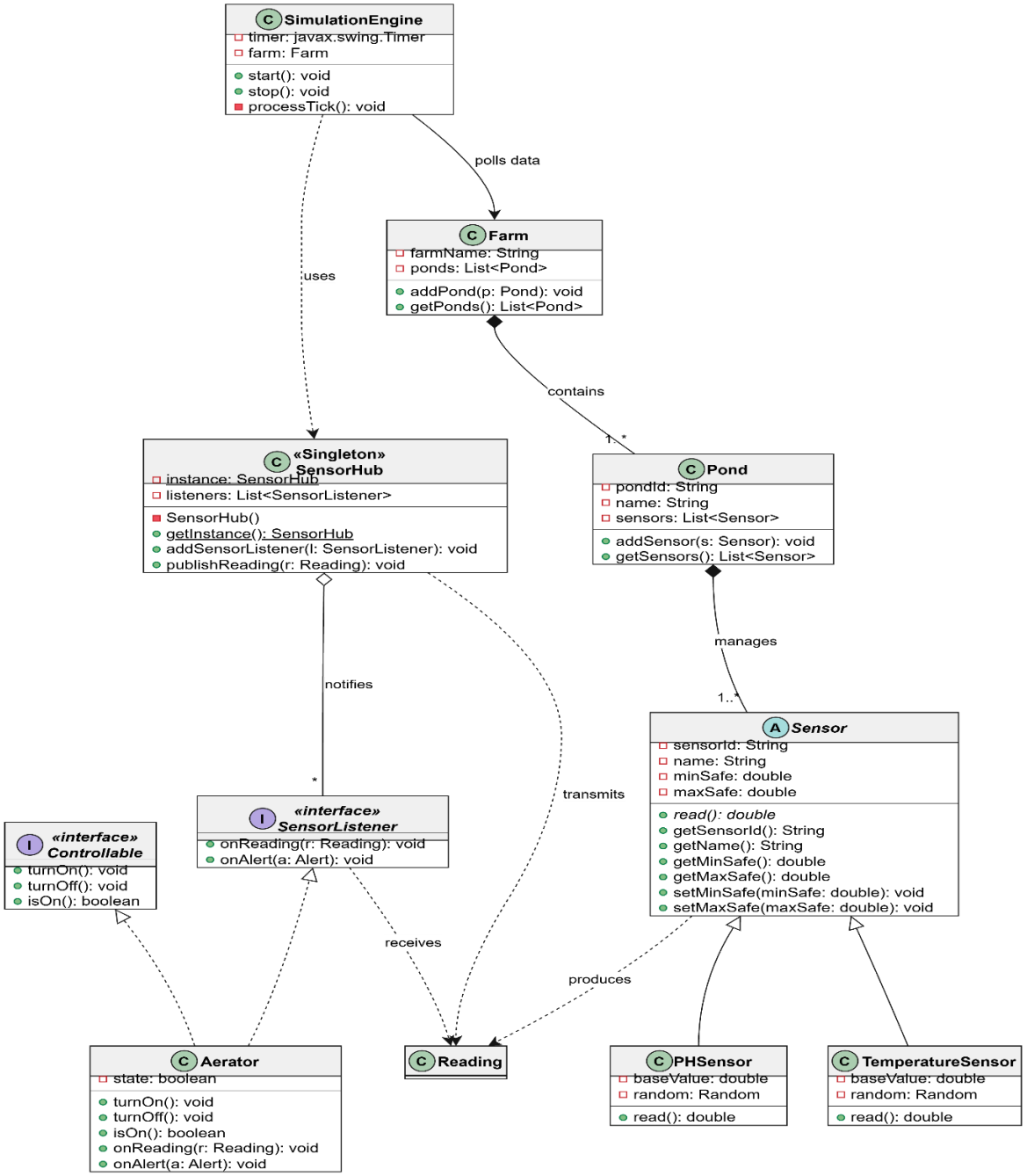
Package tương đối	Các lớp và trách nhiệm
model	Sensor, bốn lớp cảm biến, Farm, Pond, Reading, Alert, SensorHub và SensorListener: dữ liệu và sự kiện.
model.device	Controllable, Aerator: giao tiếp và mô hình thiết bị chấp hành.
service / util	SimulationEngine điều phối lấy mẫu; PropertiesHelper đọc và ghi cấu hình.
common / exception	BaseWindow cung cấp khung cửa sổ; InvalidThresholdException biểu diễn ngưỡng không hợp lệ.

Package tương đối	Các lớp và trách nhiệm
ui và các package con	MainWindow; panels gồm DashboardPanel, GaugePanel; dialogs gồm ThresholdDialog, AboutDialog; tables gồm LogTableModel; charts gồm ChartWindow.

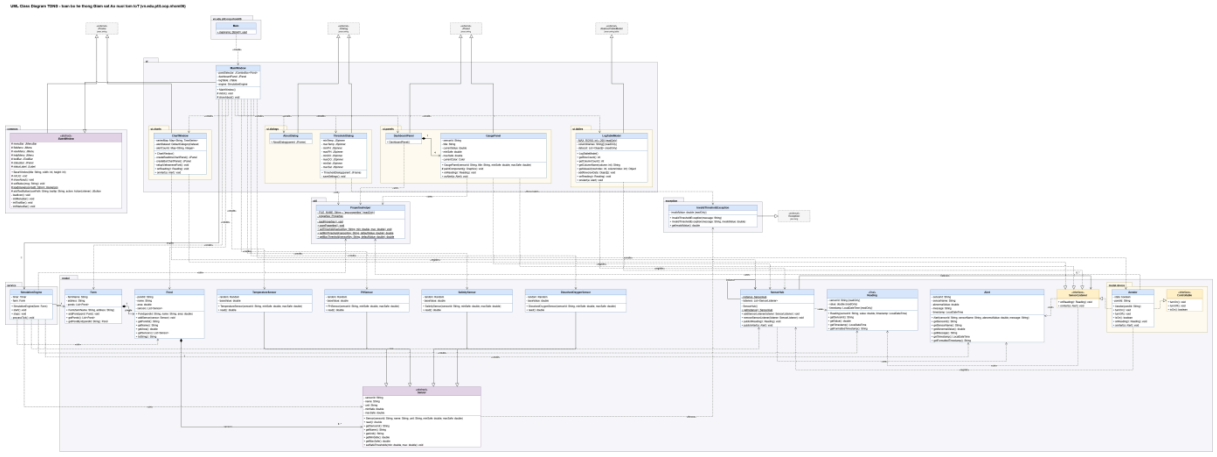
2.3. Mô hình lớp và các quan hệ thiết kế

Tên Lớp / Interface	Thuộc tính (Attributes)	Phương thức (Methods)	Vai trò & Trách nhiệm
Sensor (Abstract Class)	- sensorId: String - name: String - minSafe: double - maxSafe: double	+ abstract read(): double + getters/setters	Khung (Template) chung cho mọi cảm biến. Đóng gói dữ liệu cơ bản.
TemperatureSensor, PHSensor	- baseValue: double - random: Random	+ read(): double	Kế thừa Sensor, cài đặt thuật toán sinh dữ liệu nhiễu Gaussian riêng.
Reading (Final Class)	- sensorId: String - value: double	+ getters()	Lớp bất biến (Immutable). Đóng gói dữ liệu đọc tại 1 thời điểm.
Pond (Class)	- pondId: String - name: String - sensors: List<Sensor>	+ addSensor(Sensor) + getSensors(): List	Đại diện ao nuôi. Quản lý vòng đời của các cảm biến bên trong nó.
Farm (Class)	- farmName: String - ponds: List<Pond>	+ addPond(Pond) + getPonds(): List	Đại diện trang trại. Quản lý danh sách các ao nuôi.
SensorListener (Interface)	Không có	+onReading(Reading) + onAlert(Alert a)	Chuẩn hóa hành vi nhận sự kiện. Bất kỳ UI nào cũng phải implement nó.
SensorHub (Singleton)	- instance:SensorHub - listeners: List<SensorListener>	+ getInstance() +addSensorListener() + publishReading()	Trung tâm phát sóng sự kiện (Publisher). Phân phối data cho các UI.
SimulationEngine	- timer: javax.swing.Timer - farm: Farm	+ start() + stop() - processTick()	Trái tim hệ thống, chạy ngầm mỗi 1 giây để đọc Sensor và đẩy cho Hub

Controllable (Interface)		+ turnOn() + turnOff() + isOn(): boolean	Ràng buộc hành vi cho các thiết bị chấp hành phần cứng.
Aerator (Class)	- state: boolean	+ turnOn(), +turnOff(), + onReading(Reading r)	Máy sục khí. Kế thừa Controllable & SensorListener để tự động chạy.



2.3.1. Sơ đồ tổng thể – kiến trúc phân lớp và tương tác

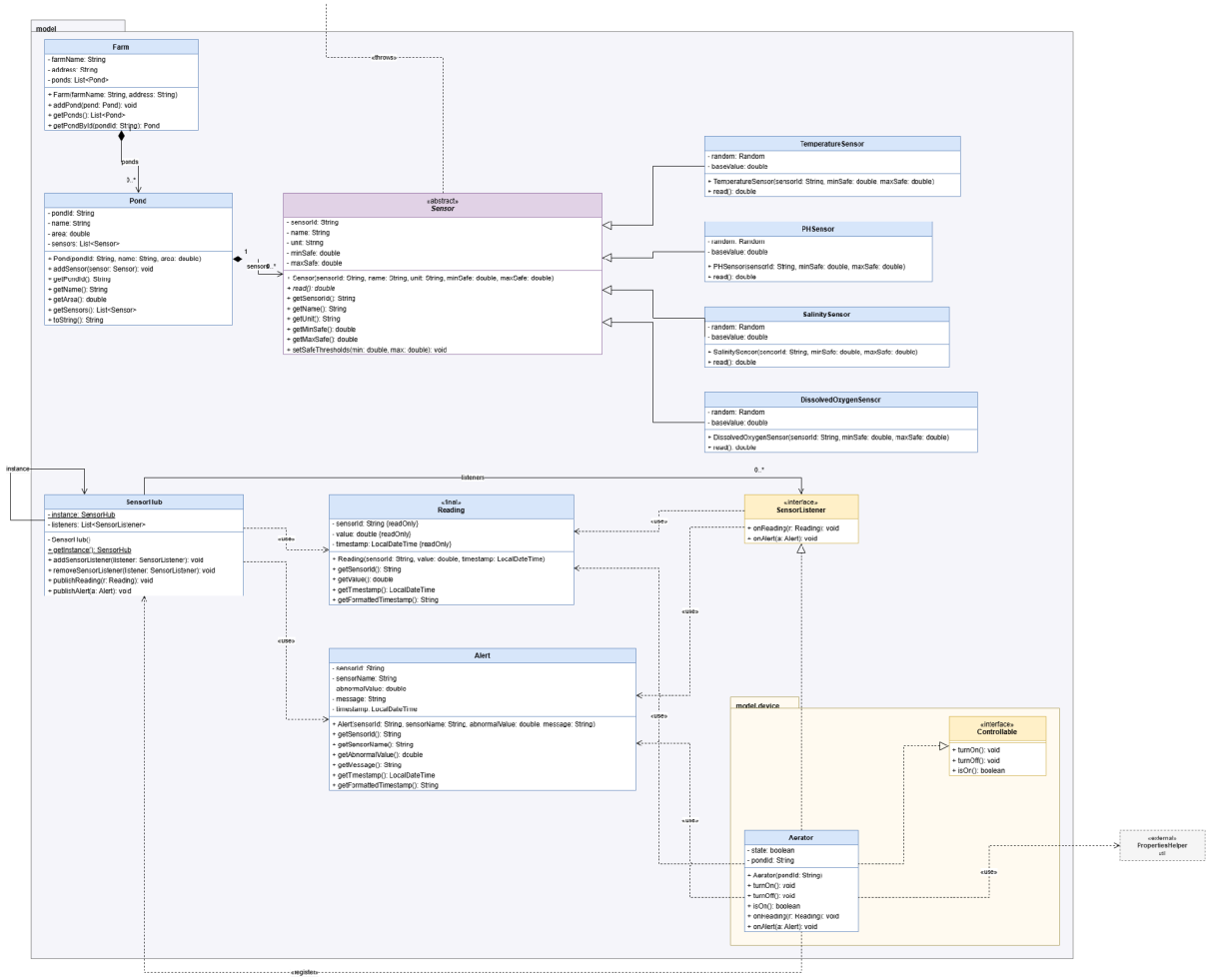


Hình UML 2.3-1. Sơ đồ lớp tổng thể của Smart Aquaculture Monitor (tệp tong.drawio).

Sơ đồ tổng thể cho thấy Main khởi chạy MainWindow; MainWindow kế thừa khung BaseWindow và phối hợp với đối tượng SimulationEngine. Tầng model quản lý Farm, Pond, các lớp Sensor, Reading, Alert và bộ phát sự kiện SensorHub; tầng ui chịu trách nhiệm trình bày đồng hồ, bảng nhật ký, hộp thoại và hai biểu đồ; tầng util cung cấp cấu hình ngưỡng. Kết nối qua SensorListener làm cho bộ phát sự kiện không phải biết trực tiếp kiểu của từng cửa sổ nhận dữ liệu.

Sơ đồ này đóng vai trò bản đồ kiến trúc. Khi cần xác định chính xác thuộc tính, phương thức và kiểu liên kết, người đọc sử dụng bảy sơ đồ con ngay sau đây. Đặc biệt, đường «creates» trong UML chỉ quan hệ khởi tạo ở chương trình, không mặc nhiên có nghĩa đối tượng được quản lý vòng đời hoặc được hủy tự động.

2.3.2. Sơ đồ model – cấu trúc nghiệp vụ, bốn cảm biến và Observer



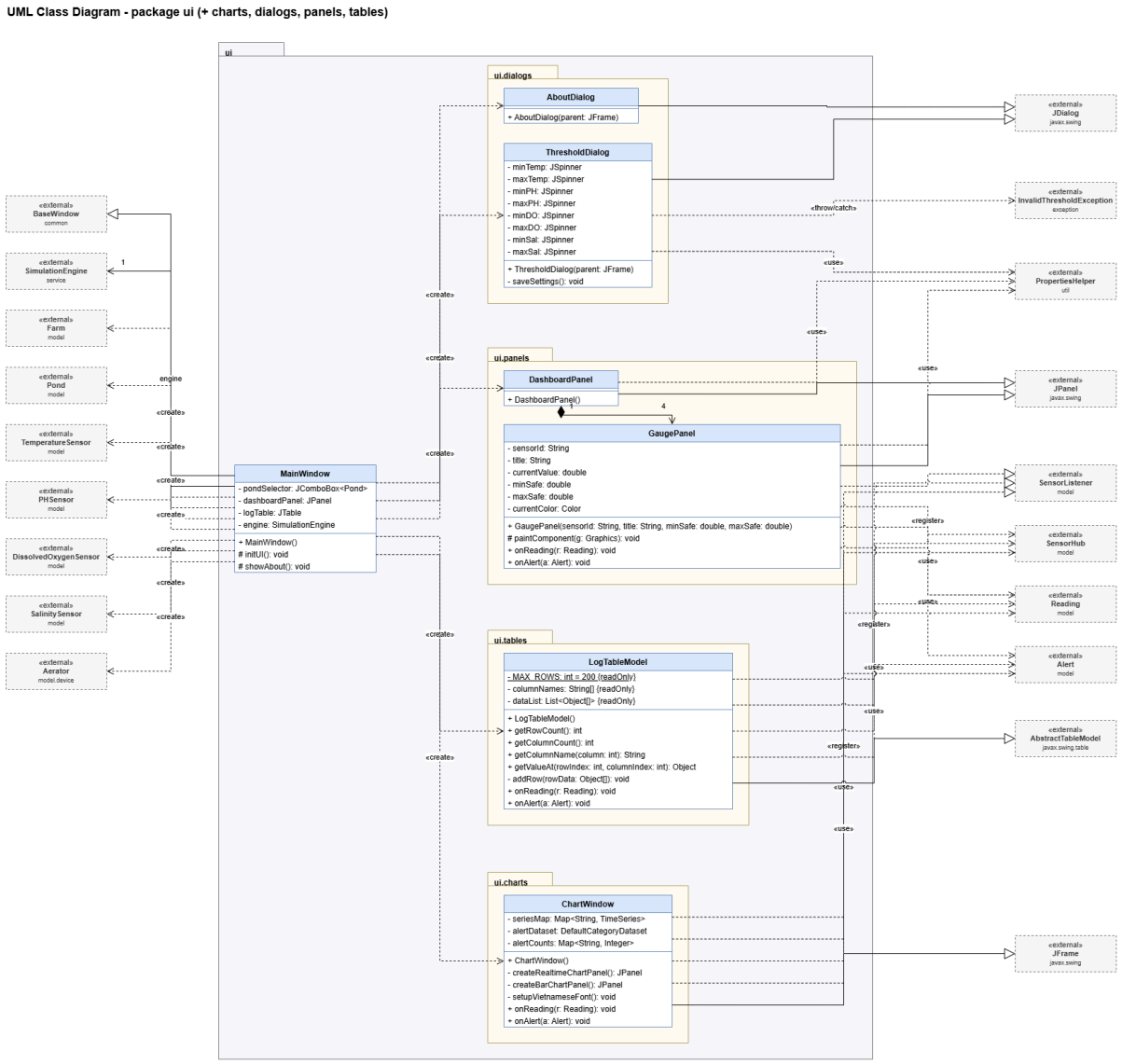
Hình UML 2.3-2. Sơ đồ lớp model và model.device.

Lớp Farm quản lý tập các Pond, còn mỗi Pond chứa List<Sensor>. Sensor là lớp trừu tượng đóng gói sensorId, name, unit và cặp ngưỡng minSafe/maxSafe; bốn lớp TemperatureSensor, PHSensor, SalinitySensor và DissolvedOxygenSensor triển khai riêng read(). Khi SimulationEngine duyệt danh sách kiểu Sensor và gọi read(), Java tự chọn phương thức của đúng lớp con. Đây là minh chứng nghiệp vụ trực tiếp cho kế thừa và đa hình động.

Reading được khai báo final, với thuộc tính private final, để biểu diễn dữ liệu của một lần đọc không bị sửa sau khi khởi tạo. SensorHub lưu danh sách SensorListener và phát Reading hoặc Alert cho các đối tượng đăng ký. Aerator vừa thực thi Controllable vừa là SensorListener để phản ứng với ngưỡng oxy hòa tan, thể hiện cách dùng nhiều interface mà không cần đa kế thừa lớp trong Java.

Lưu ý khi diễn giải quan hệ: dấu hình thoi Farm–Pond và Pond–Sensor mô tả cấu trúc chứa theo nghiệp vụ; mã Java hiện nhận các đối tượng được tạo ở bên ngoài qua addPond()/addSensor(), chưa ràng buộc quyền sở hữu duy nhất hay cơ chế hủy theo vòng đời. Vì vậy không nên dùng sơ đồ để khẳng định rằng việc xóa Farm sẽ tự động hủy các đối tượng Sensor. Ngoài những phương thức được ghi trong hình, Sensor ở code hiện còn có setMinSafe(), setMaxSafe(), isSafe(), isAlert() và toString().

2.3.3. Sơ đồ ui – phân tách hiển thị, tương tác và dữ liệu

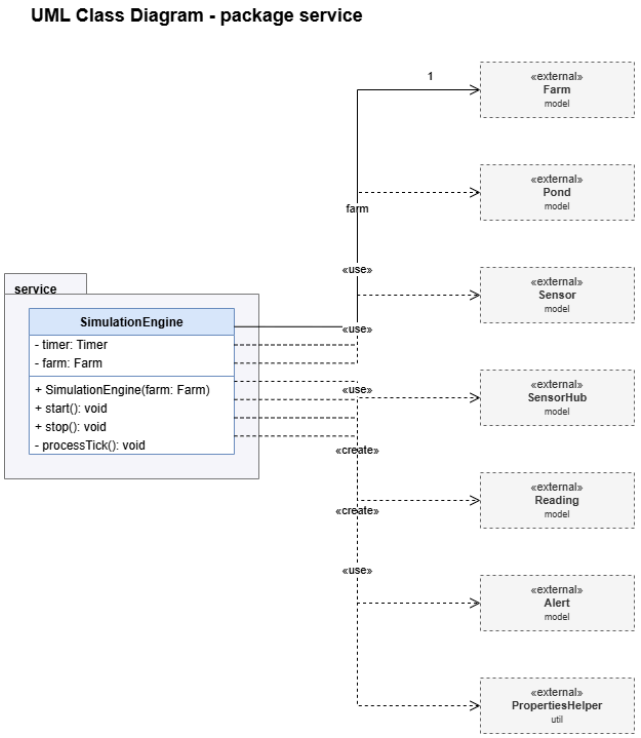


Hình UML 2.3-3. Sơ đồ lớp ui và các package charts, dialogs, panels, tables.

MainWindow mở ThresholdDialog và ChartWindow, đồng thời chứa DashboardPanel, bảng log và bộ chọn ao. DashboardPanel tạo bốn GaugePanel, mỗi panel quản lý dữ liệu hiển thị của một loại cảm biến và ghi đè paintComponent(Graphics) để vẽ bằng Graphics2D. LogTableModel kế thừa AbstractTableModel để cung cấp dữ liệu cho JTable; ChartWindow nhận sự kiện rồi cập nhật TimeSeries và DefaultCategoryDataset của JFreeChart.

Trong mô hình Observer, GaugePanel, LogTableModel và ChartWindow thực thi SensorListener nhưng có hành vi phản hồi khác nhau: cập nhật màu/giá trị đồng hồ, thêm dòng nhật ký, hoặc thay đổi biểu đồ. Hình UML phản ánh mối liên kết này nhưng không đồng nghĩa giao diện đã hỗ trợ chuyển đổi giám sát nhiều ao. Hiện JComboBox mới nạp danh sách ao mẫu, chưa có luồng cập nhật dashboard theo ao được chọn. Ngoài ra, ChartWindow có đăng ký listener nhưng mã nguồn chưa hủy đăng ký khi đóng cửa sổ.

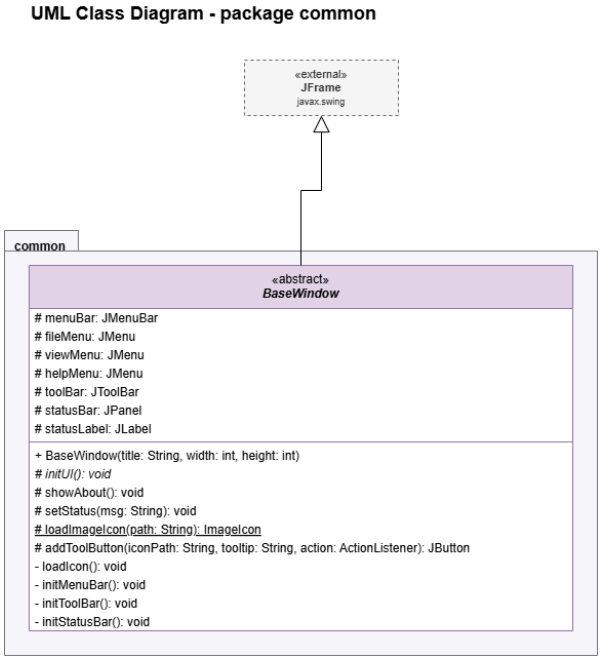
2.3.4. Sơ đồ service – điều phối lấy mẫu và phát cảnh báo



Hình UML 2.3-4. Sơ đồ lớp SimulationEngine trong package service.

SimulationEngine giữ tham chiếu đến Farm và javax.swing.Timer. Mỗi nhịp của timer, processTick() duyệt các Pond và Sensor, gọi read(), tạo Reading rồi phát sự kiện qua SensorHub; nếu giá trị nằm ngoài cặp ngưỡng, engine phát thêm Alert. Việc chỉ phụ thuộc vào kiểu trừu tượng Sensor giúp engine không cần chứa nhánh xử lý riêng cho bốn loại cảm biến. Chu kỳ Timer 1000 ms là chu kỳ mô phỏng được yêu cầu, không phải thời gian thực cứng; callback Swing Timer hoạt động trên Event Dispatch Thread.

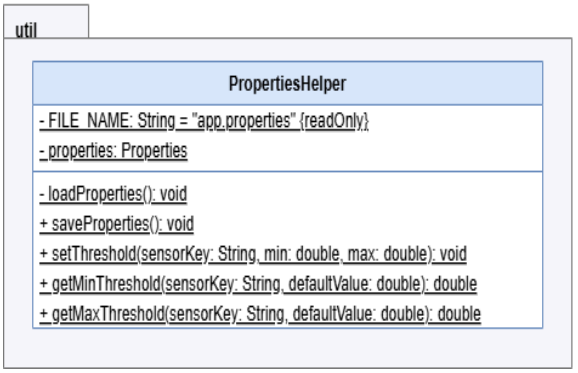
2.3.5. Sơ đồ common – lớp khung BaseWindow



Hình UML 2.3-5. Sơ đồ BaseWindow kế thừa JFrame.

BaseWindow chuẩn hóa menu, toolbar, status bar và phương thức tiện ích; MainWindow kế thừa các thành phần protected và bắt buộc cài đặt initUI(). Quan hệ kế thừa được thể hiện bằng mũi tên hướng từ lớp con tới lớp cha. Khi đối chiếu mã nguồn cần lưu ý constructor BaseWindow gọi initUI() trước khi constructor của MainWindow kết thúc; các trường UI phụ thuộc vào initUI() phải được khởi tạo đúng thời điểm để tránh giá trị null hoặc bị gán lại.

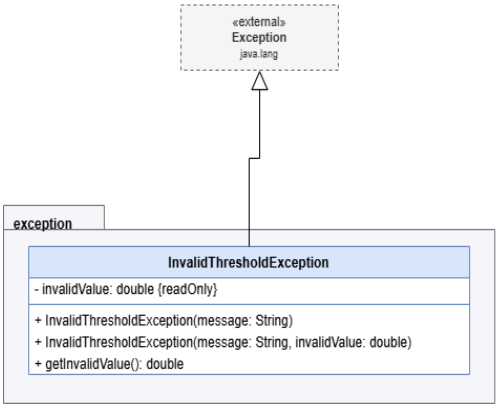
2.3.6. Sơ đồ util – quản lý cấu hình ngưỡng



Hình UML 2.3-6. Sơ đồ lớp PropertiesHelper trong package util.

PropertiesHelper cung cấp các phương thức static để tải, truy xuất và lưu cấu hình từ app.properties. Ngưỡng được tra cứu theo mã cảm biến temp, ph, do và sal; ThresholdDialog gọi setThreshold() và saveProperties() sau khi kiểm tra dữ liệu. Điểm cần phân biệt là luồng lưu của hộp thoại thực hiện kiểm tra riêng, chưa trực tiếp gọi Sensor.setSafeThresholds(); do đó đây là hai điểm kiểm soát logic có liên quan nhưng chưa được hợp nhất thành một đường gọi nghiệp vụ duy nhất.

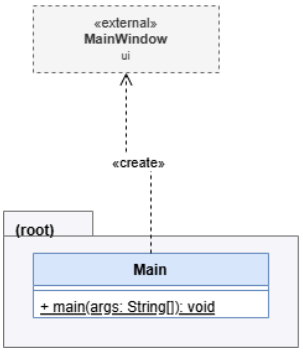
2.3.7. Sơ đồ exception – ngoại lệ nghiệp vụ tự định nghĩa



Hình UML 2.3-7. Sơ đồ InvalidThresholdException kế thừa Exception.

InvalidThresholdException là checked exception với hai hàm tạo nạp chồng: một nhận chuỗi thông báo và một nhận thêm invalidValue. Trong ThresholdDialog, trường hợp Min lớn hơn hoặc bằng Max tạo ngoại lệ và được bắt bằng try-catch để hiển thị thông báo. Ở Sensor, setSafeThresholds() cũng ném ngoại lệ khi min >= max; mã hiện chưa áp dụng đồng nhất quy tắc từ chối số âm, NaN hoặc vô cực trong mọi trường hợp. Vì vậy sơ đồ chứng minh có cấu trúc ngoại lệ tự định nghĩa, không đồng nghĩa toàn bộ đầu vào đã được xác thực hoàn chỉnh.

2.3.8. Sơ đồ lớp Main – điểm vào chương trình



Hình UML 2.3-8. Sơ đồ lớp Main và quan hệ khởi tạo MainWindow.

Main cung cấp main(String[] args) và thực hiện SwingUtilities.invokeLater(...) để tạo, hiển thị MainWindow trên Event Dispatch Thread. Đây là trách nhiệm khởi chạy giao diện, khác với trách nhiệm điều phối đọc cảm biến của SimulationEngine. Việc tách hai vai trò giúp giảm mức độ ghép nối giữa điểm vào chương trình và logic mô phỏng.

2.4. Luận giải quyết định thiết kế

Sensor được chọn làm lớp trừu tượng vì các cảm biến dùng chung trạng thái và cách cập nhật ngưỡng, nhưng có thuật toán đọc khác nhau. read() là điểm mở rộng; vòng lặp trong SimulationEngine chỉ cần làm việc với kiểu Sensor.

Reading là lớp final, có các trường private final, chỉ cung cấp getter. String và LocalDateTime cũng là các kiểu bất biến, vì vậy dữ liệu của một lần đo không thay đổi sau khi tạo. Tính chất này bảo vệ bản ghi; nó không tự làm cho toàn bộ SensorHub hoặc giao diện trở thành an toàn đa luồng.

SensorHub áp dụng Observer để giảm phụ thuộc giữa bộ sinh dữ liệu và nơi sử dụng dữ liệu. Hub đồng thời dùng Singleton với constructor private và getInstance(). Bản cài đặt hiện tại không đồng bộ hóa truy cập đa luồng; luồng xử lý mô phỏng chủ yếu hoạt động trên EDT của Swing.

Farm chứa Pond và Pond chứa Sensor thể hiện cấu trúc chứa theo miền nghiệp vụ. Tuy nhiên, các đối tượng được tạo bên ngoài rồi thêm vào danh sách; mã nguồn chưa ràng buộc sở hữu duy nhất hoặc vòng đời hủy đối tượng. Cấu trúc này chưa phải Composite Pattern đầy đủ vì Farm, Pond và Sensor không dùng chung giao diện thành phần để xử lý đệ quy.

A. Vì sao sử dụng Lớp trừu tượng (Abstract Class) cho Sensor?

Tái sử dụng mã: Các cảm biến (Nhiệt độ, pH, DO...) đều chia sẻ chung các thuộc tính như sensorId, name, minSafe, maxSafe và các phương thức getter/setter. Nếu dùng Interface, ta sẽ phải viết lại code khai báo các biến này ở từng lớp con, gây dư thừa (vi phạm nguyên tắc DRY). Lớp trừu tượng giúp gom toàn bộ code chung vào một chỗ.

Bắt buộc Đa hình: Lớp Sensor định nghĩa phương thức abstract double read(); nhưng không viết thân hàm. Điều này ép buộc 100% các lớp con (như PHSensor, TemperatureSensor) phải tự Ghi đè (Override) hàm read() để định nghĩa thuật toán sinh dữ liệu riêng (ví dụ DO thì có xác suất tụt mạnh, Nhiệt độ thì dao động đều).

Vòng lặp trong SimulationEngine chỉ cần gọi sensor.read(), tính đa hình sẽ tự động kích hoạt hàm tương ứng của lớp con.

B. Vì sao sử dụng Interface cho SensorListener và Controllable?

Khắc phục hạn chế Đơn kế thừa của Java: Trong Java, một lớp chỉ được extends (kế thừa) duy nhất một lớp cha. Lớp GaugePanel đã bắt buộc phải extends JPanel để vẽ giao diện, lớp ChartWindow phải extends JFrame. Nếu SensorListener là một abstract class, các UI này sẽ không thể kế thừa nó được nữa. Việc sử dụng interface SensorListener cho phép các thành phần UI vừa giữ được bản chất đồ họa, vừa mang thêm "nhân dạng" của một Quan sát viên (Observer).

Giảm độ phụ thuộc: Lớp SensorHub chỉ quan tâm đẩy dữ liệu cho những ai có mác SensorListener, nó hoàn toàn không cần biết đối tượng nhận là một cái Bảng (Table), một cái Đồng hồ (Gauge) hay một Thiết bị sục khí (Aerator).

C. Vì sao sử dụng Composition (Kết hợp) thay vì Aggregation (Tập hợp)?

Mô phỏng thực tế vòng đời: Sơ đồ hệ thống sử dụng Composition 2 tầng: Farm → Pond → Sensor. Đây là quan hệ "Bao hàm chặt chẽ" (Has-A). Nếu một ao nuôi (Pond) bị xóa bỏ, thì toàn bộ cảm biến (Sensor) cắm dưới ao đó cũng trở nên vô nghĩa và bị hủy theo.

Bảo vệ toàn vẹn Đóng gói: Để bảo vệ cấu trúc Composition này, trong lớp Pond, hàm getSensors() không bao giờ trả về trực tiếp mảng bộ nhớ gốc (return this.sensors), mà sử dụng return java.util.Collections.unmodifiableList(sensors). Quyết định thiết kế này giúp khóa chặt mảng nội bộ, ngăn chặn mọi nỗ lực add/remove cảm biến trái phép từ các lớp bên ngoài mà không thông qua hàm addSensor() chính thức của lớp Pond.

CHƯƠNG 3. CÀI ĐẶT VÀ ÁNH XẠ NGUYÊN LÝ OOP

3.1. Ánh xạ nguyên lý OOP vào mã nguồn

Nguyên lý	Vị trí thể hiện	Ý nghĩa
Trừu tượng	Sensor.read(); BaseWindow.initUI(); SensorListener; Controllable.	Tách biệt hợp đồng giao tiếp (Contract) khỏi phần cài đặt chi tiết; cho phép đa dạng hóa nguồn dữ liệu và thiết bị ngoại vi.

Nguyên lý	Vị trí thể hiện	Ý nghĩa
Đóng gói	Trường private của Sensor; setSafeThresholds(); Reading bất biến.	Ngăn chặn truy cập bất hợp thức từ bên ngoài, kiểm soát tính toàn vẹn của dữ liệu và ngưỡng an toàn.
Kế thừa	Bốn cảm biến extends Sensor; MainWindow extends BaseWindow.	Tái sử dụng trạng thái, hành vi đo lường và khung giao diện chuẩn hóa của hệ thống.
Đa hình	sensor.read() trong processTick(); listener.onReading(); GaugePanel.paintComponent().	Cho phép lời gọi phương thức tự động thực thi đúng mã nguồn tương ứng theo kiểu đối tượng thực tế tại thời điểm chạy (Runtime).
Tính Kết hợp	• Cấu trúc 2 tầng: Farm chứa List<Pond>, và Pond chứa List<Sensor> • MainWindow chứa DashboardPanel, JTable, SimulationEngine • DashboardPanel chứa 4 đối tượng GaugePanel	Thể hiện mối quan hệ "Has-A" phản ánh chính xác cấu trúc thực tế của trang trại nuôi tôm; các đối tượng thành phần kết hợp chặt chẽ tạo nên một thể thống nhất và ràng buộc về vòng đời nghiệp vụ
Xử lý ngoại lệ	• Lớp ngoại lệ tự định nghĩa: InvalidThresholdException kế thừa Exception • Cơ chế ném ngoại lệ (throw) tại Sensor.setSafeThresholds() • Khối try-catch bắt lỗi tại ThresholdDialog.saveSettings() và cảnh báo qua JOptionPane	Bắt lỗi vi phạm nghiệp vụ ($Min \geq Max$), ngăn chặn ghi dữ liệu rác vào file cấu hình, đồng thời giúp phần mềm vận hành an toàn (Fail-safe), không bao giờ bị tắt đột ngột (crash) do người dùng nhập sai.
Trực quan hóa dữ liệu	• Tự vẽ đồ họa nâng cao: Dùng Graphics2D vẽ kim đo và cung tròn đổi màu Xanh/Đỏ trong GaugePanel .Tích hợp thư viện ngoại vi **JFreeChart 1.5.x**: TimeSeriesChart (biến động 60s) và BarChart (thống kê số lần cảnh báo)	Chuyển đổi các con số đo lường khô khan thành hình ảnh trực quan động, giúp người quản lý ao nuôi nhận biết ngay lập tức trạng thái an toàn hay nguy hiểm theo thời gian thực.

Nguyên lý	Vị trí thể hiện	Ý nghĩa
Xử lý sự kiện	• Mô hình **Observer Pattern**: SensorHub (Subject) phát sự kiện tới các SensorListener (Observer) • Bộ định thời javax.swing.Timer (chu kỳ 1000ms) phát sự kiện định kỳ • ActionListener xử lý sự kiện bấm nút trên Menu, Toolbar và Dialog	Giảm thiểu tối đa sự phụ thuộc (Loose Coupling) giữa luồng sinh dữ liệu và giao diện; đồng bộ hóa việc cập nhật dữ liệu tự động lên EDT (Event Dispatch Thread)

3.2. Cài đặt cảm biến và sinh dữ liệu mô phỏng

Các constructor của bốn cảm biến đều nhận **sensorId, minSafe và maxSafe**. MainWindow tạo các mã temp, ph, do và sal. Ngưỡng khởi tạo lần lượt là 26–32 °C, 7,5–8,5 pH, 4–10 mg/L và 10–25 %; ngưỡng này mô tả cấu hình của bản mô phỏng.

Cảm biến	Công thức trước làm tròn	Xử lý bổ sung
Nhiệt độ	29,0 + nextGaussian() × 1,5	Làm tròn một chữ số thập phân.
pH	8,0 + nextGaussian() × 0,4	Làm tròn hai chữ số thập phân.
Oxy hòa tan	5,0 + nextGaussian() × 1,2	Có xác suất 5% trừ thêm 3,0; chặn dưới tại 0; làm tròn một chữ số.
Độ mặn	15,0 + nextGaussian() × 0,8	Chặn dưới tại 0; làm tròn một chữ số.

Trích đoạn read() trong model/TemperatureSensor.java:

```java
package vn.edu.ptit.oop.nhom06.model;

import java.util.Random;


public class TemperatureSensor extends Sensor {

    private Random random;
    private double baseValue;

    public TemperatureSensor(String sensorId, double minSafe, double maxSafe) {
        super(sensorId, "Nhiệt độ (Temperature)", "°C", minSafe, maxSafe);
        this.random = new Random();
        this.baseValue = 29.0;
    }


    @Override
    public double read() {

        double noise = random.nextGaussian() * 1.5;
        double currentValue = this.baseValue + noise;

        return Math.round(currentValue * 10.0) / 10.0;
    }
}
```

Mỗi lần gọi read() sinh một mẫu mới quanh giá trị trung tâm. Mô phỏng không sử dụng giá trị trước để tính giá trị sau, nên không biểu diễn đầy đủ diễn biến vật lý liên tục của ao nuôi. Dữ liệu dùng để minh họa giao diện và cơ chế cảnh báo.

Tính Đóng gói trong lớp Pond (Trả về unmodifiable list)

```java
package vn.edu.ptit.oop.nhom06.model;

import java.util.ArrayList;
import java.util.List;

public class Pond {
    private String pondId;
    private String name;
    private double area;

    private List<Sensor> sensors;

    public Pond(String pondId, String name, double area) {
        this.pondId = pondId;
        this.name = name;
        this.area = area;
        this.sensors = new ArrayList<>();
    }

    public void addSensor(Sensor sensor) {
        if (sensor != null) {
            this.sensors.add(sensor);
        }
    }

    public String getPondId() { return pondId; }
    public String getName() { return name; }
    public double getArea() { return area; }

    public List<Sensor> getSensors() {
        return java.util.Collections.unmodifiableList(sensors);
    }

    @Override
    public String toString() {
        return name;
    }
}

```

Đoạn mã trong lớp Pond thể hiện tính Đóng gói chặt chẽ qua việc giấu kín dữ liệu bằng private và chỉ cung cấp các phương thức getter an toàn để đọc thông tin. Đáng chú ý, phương thức getSensors() sử dụng Collections.unmodifiableList(sensors) nhằm trả về danh sách chỉ đọc, ngăn chặn hoàn toàn việc bên ngoài tự ý thêm hay xóa cảm biến làm sai lệch dữ liệu. Đồng thời, phương thức toString() được ghi đè để trả về tên ao, giúp các thành phần giao diện như JComboBox hiển thị tên trực quan thay vì mã băm bộ nhớ mặc định

Lớp bất biến (Immutable Class).

```java
package vn.edu.ptit.oop.nhom06.model;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;


public final class Reading {

    private final String sensorId;
    private final double value;
    private final LocalDateTime timestamp;

    public Reading(String sensorId, double value, LocalDateTime timestamp) {
        this.sensorId = sensorId;
        this.value = value;
        this.timestamp = timestamp;
    }


    public String getSensorId() {
        return sensorId;
    }

    public double getValue() {
        return value;
    }

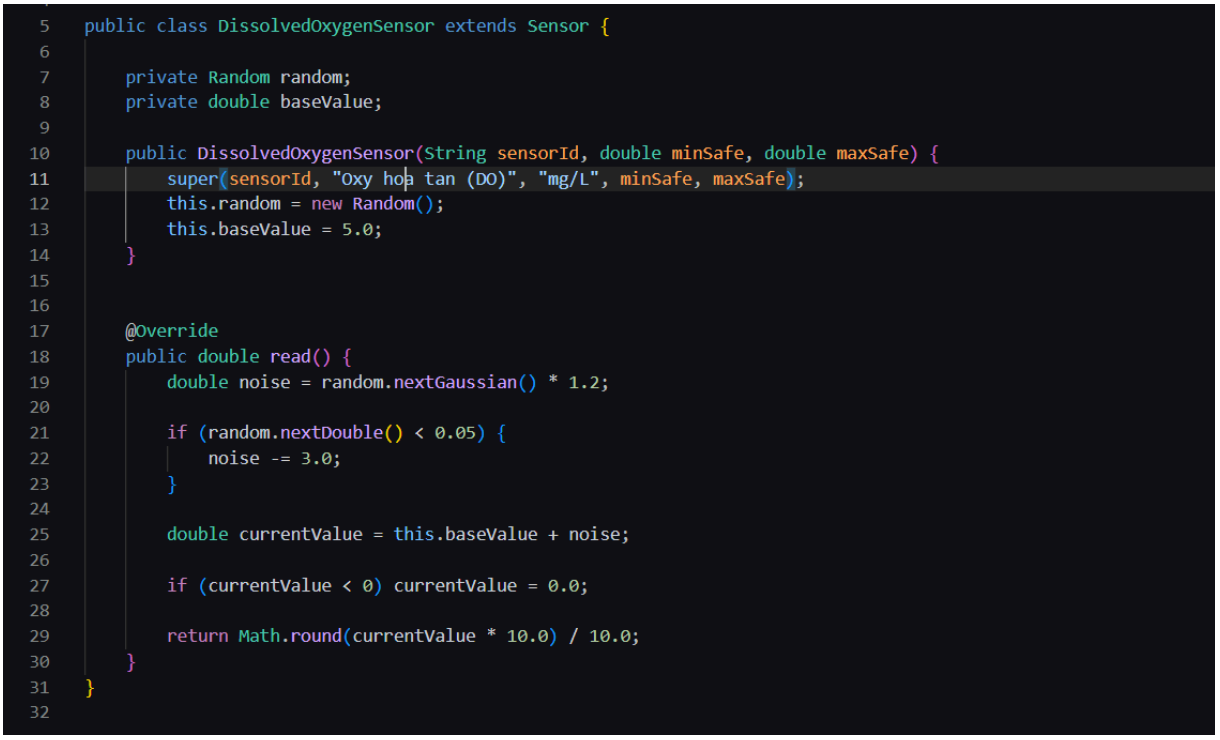
    public LocalDateTime getTimestamp() {
        return timestamp;
    }



    public String getFormattedTimestamp() {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("HH:mm:ss dd/MM/yyyy");
```

Lớp Reading được thiết kế theo mô hình Lớp bất biến (Immutable Class) nhằm bảo vệ tính toàn vẹn tuyệt đối của dữ liệu đo đạc. Lớp sử dụng từ khóa final để ngăn chặn việc kế thừa, đồng thời toàn bộ thuộc tính đều là private final chỉ được gán duy nhất một lần qua constructor. Việc loại bỏ hoàn toàn các phương thức setter và chỉ cung cấp getter đảm bảo mỗi bản ghi sau khi tạo ra sẽ không thể bị thay đổi giá trị hay làm sai lệch lịch sử bởi bất kỳ thành phần nào khác trong hệ thống.

Tính Đa hình nghiệp vụ trong DissolvedOxygenSensor.java:



```java
public class DissolvedOxygenSensor extends Sensor {

    private Random random;
    private double baseValue;

    public DissolvedOxygenSensor(String sensorId, double minSafe, double maxSafe) {
        super(sensorId, "Oxy hoa tan (DO)", "mg/L", minSafe, maxSafe);
        this.random = new Random();
        this.baseValue = 5.0;
    }


    @Override
    public double read() {
        double noise = random.nextGaussian() * 1.2;

        if (random.nextDouble() < 0.05) {
            noise -= 3.0;
        }

        double currentValue = this.baseValue + noise;

        if (currentValue < 0) currentValue = 0.0;

        return Math.round(currentValue * 10.0) / 10.0;
    }
}
```

Đoạn mã trong lớp DissolvedOxygenSensor thể hiện rõ nét tính Đa hình nghiệp vụ (Polymorphism) thông qua việc ghi đè phương thức trừu tượng read() từ lớp cha Sensor. Bên cạnh việc mô phỏng nồng độ oxy hòa tan dao động quanh giá trị cơ sở 5,0 mg/L bằng hàm phân phối chuẩn Gauss (nextGaussian() * 1.2), phương thức được cài đặt logic đặc thù với 5% xác suất sụt giảm đột ngột 3,0 mg/L. Đây là cơ chế mô phỏng sự cố môi trường có chủ đích, trực tiếp kéo giá trị DO xuống dưới ngưỡng an toàn tối thiểu (4,0 mg/L) để kích hoạt chuỗi phản ứng của toàn hệ thống: đồng hồ GaugePanel chuyển đỏ, nhật ký phát sinh bản tin cảnh báo Alert, và máy sục khí Aerator tự động bật ở tính năng BONUS. Đồng thời, thuật toán cũng chủ động chặn dưới tại 0,0 và làm tròn một chữ số thập phân nhằm đảm bảo dữ liệu vật lý sinh ra luôn có ý nghĩa thực tế.

Tính Đa hình đồ họa giao diện trong GaugePanel.java:

```
14  public class GaugePanel extends JPanel implements SensorListener {
22      public GaugePanel(String sensorId, String title, double minSafe, double maxSafe) {
31          SensorHub.getInstance().addSensorListener(this);
32      }
33
34
35
36      @Override
37      protected void paintComponent(Graphics g) {
38          super.paintComponent(g);
39
40          Graphics2D g2d = (Graphics2D) g;
41
42          g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
43
44          int width = getWidth();
45          int height = getHeight();
46
47          g2d.setColor(new Color(220, 220, 220));
48          g2d.setStroke(new BasicStroke(20, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
49
50          g2d.drawArc(40, 40, width - 80, height * 2 - 120, 0, 180);
51
52          double range = (maxSafe - minSafe) * 2;
53          double displayMin = minSafe - (range * 0.25);
54          double displayMax = maxSafe + (range * 0.25);
55
56          double pct = (currentValue - displayMin) / (displayMax - displayMin);
57          if (pct < 0) pct = 0;
58          if (pct > 1) pct = 1;
59
60          int angle = (int) (180 * pct);
61
62          g2d.setColor(currentColor);
63          g2d.drawArc(40, 40, width - 80, height * 2 - 120, 180, -angle);
64
65          g2d.setFont(new Font("Segoe UI", Font.BOLD, 28));
66          String valStr = String.format("%.1f", currentValue);
67          FontMetrics fm = g2d.getFontMetrics();
68          int textWidth = fm.stringWidth(valStr);
69          g2d.setColor(Color.BLACK);
70          g2d.drawString(valStr, (width - textWidth) / 2, height - 30);
71
72          g2d.setFont(new Font("Segoe UI", Font.PLAIN, 16));
73          g2d.setColor(Color.DARK_GRAY);
74          int titleWidth = g2d.getFontMetrics().stringWidth(title);
75          g2d.drawString(title, (width - titleWidth) / 2, height - 70);
76      }
77
```

Đoạn mã trong lớp GaugePanel thể hiện tính Đa hình (Polymorphism) trên tầng giao diện thông qua việc kế thừa JPanel và ghi đè phương thức paintComponent(Graphics g). Thay vì sử dụng giao diện mặc định, phương thức tận dụng đối tượng Graphics2D với tính năng khử răng cưa để tự vẽ đồng hồ bán nguyệt 180°. Thuật toán tính toán góc quay (angle) dựa trên tỷ lệ giữa giá trị tức thời (currentValue) và dải đo an toàn, đồng thời thay đổi màu sắc cung đo (currentColor) sang Xanh hoặc Đỏ để chỉ thị trực quan tình trạng môi trường. Bên cạnh đó, việc sử dụng FontMetrics giúp giá trị số và tiêu đề cảm biến luôn được căn giữa cân đối trên bảng điều khiển.

3.3. Đóng gói và xử lý ngoại lệ

Sensor.setSafeThresholds() kiểm tra min >= max trước khi gán hai trường ngưỡng. Khi vi phạm, phương thức ném InvalidThresholdException và giữ nguyên ngưỡng cũ. Constructor Sensor hiện gán trực tiếp các ngưỡng, chưa dùng lại bước kiểm tra này.

InvalidThresholdException kế thừa Exception và có hai constructor. Constructor một tham số nhận thông điệp, đặt invalidValue bằng 0,0; constructor hai tham số cho phép lưu giá trị vi phạm. Các vị trí ném ngoại lệ đang dùng constructor một tham số.

```
30
31      public void setSafeThresholds(double min, double max) throws InvalidThresholdException {
32          if (min >= max) {
33              throw new InvalidThresholdException(
34                  "Loi cau hinh cam bien [" + this.name + "]: Nguong Min (" + min + ") phai nho hon Max (" + max + ")"
35              );
36          }
37          this.minSafe = min;
38          this.maxSafe = max;
39      }
40  }
```

Trong ThresholdDialog, saveSettings() đọc bốn giá trị từ JSpinner, tự kiểm tra cặp ngưỡng nhiệt độ và pH, rồi gọi PropertiesHelper.setThreshold(). Hộp thoại bắt InvalidThresholdException và hiển thị JOptionPane. Luồng hiện tại chưa gọi Sensor.setSafeThresholds(), nên chưa tạo thành chuỗi gọi trực tiếp từ giao diện đến setter của mô hình.

Khối try-catch xác thực trên giao diện:

```
82
83          try {
84
85
86              if (currentMinTemp >= currentMaxTemp) {
87                  throw new InvalidThresholdException("Nguong toi thieu cua Nhiet do phai nho hon nguong toi da!");
88              }
89              if (currentMinPH >= currentMaxPH) {
90                  throw new InvalidThresholdException("Nguong toi thieu cua pH phai nho hon nguong toi da!");
91              }
92              if (currentMinDO >= currentMaxDO) {
93                  throw new InvalidThresholdException("Nguong Min cua Oxy phai nho hon Max!");
94              }
95              if (currentMinSal >= currentMaxSal) {
96                  throw new InvalidThresholdException("Nguong Min cua do man phai nho hon Max!");
97              }
98
99              PropertiesHelper.setThreshold("temp", currentMinTemp, currentMaxTemp);
100             PropertiesHelper.setThreshold("ph", currentMinPH, currentMaxPH);
101             PropertiesHelper.setThreshold("do", currentMinDO, currentMaxDO);
102             PropertiesHelper.setThreshold("sal", currentMinSal, currentMaxSal);
103
104             JOptionPane.showMessageDialog(this, "Luu cau hinh thanh cong!", "Thong bao",
105                     JOptionPane.INFORMATION_MESSAGE);
106             dispose();
107
108         } catch (InvalidThresholdException ex) {
109
110             JOptionPane.showMessageDialog(this, ex.getMessage(), "Loi nhap lieu", JOptionPane.ERROR_MESSAGE);
111         }
112     }
```

Đoạn mã trên thể hiện cơ chế kiểm tra dữ liệu và xử lý ngoại lệ khi người dùng nhấn nút lưu cấu hình trong ThresholdDialog. Trước khi ghi nhận dữ liệu, chương trình kiểm tra điều kiện hợp lệ cho cả 4 loại cảm biến: ngưỡng tối thiểu (Min) bắt buộc phải nhỏ hơn ngưỡng tối đa (Max). Nếu phát hiện người dùng nhập sai logic, lệnh throw sẽ chủ động kích hoạt ngoại lệ tự định nghĩa InvalidThresholdException để dừng ngay tiến trình, ngăn việc ghi dữ liệu sai vào file app.properties. Khối catch ngay bên dưới sẽ đón bắt lỗi này và hiển thị hộp thoại JOptionPane thông báo rõ nguyên nhân cho người dùng. Cách cài đặt này giúp bảo vệ tính toàn vẹn của dữ liệu cấu hình, đồng thời giữ ứng dụng luôn an toàn, không bị tắt đột ngột (crash) khi gặp lỗi nhập liệu từ người dùng

3.4. Bộ định thời và cơ chế Observer

Main.main() dùng SwingUtilities.invokeLater() để tạo cửa sổ trên EDT. SimulationEngine dùng new Timer(1000, e -> processTick()); callback của Swing Timer cũng chạy trên EDT. Khoảng 1000 ms là chu kỳ yêu cầu, không phải cam kết thời gian thực tuyệt đối khi luồng giao diện bận.

Trích đoạn thực tế trong service/SimulationEngine.java:

```
9   public class SimulationEngine {
26      public void stop() {
27          if (timer.isRunning()) {
28              timer.stop();
29          }
30      }
31
32      private void processTick() {
33
34          for (Pond pond : farm.getPonds()) {
35
36              for (Sensor sensor : pond.getSensors()) {
37
38                  double value = sensor.read();
39
40                  Reading reading = new Reading(sensor.getSensorId(), value, LocalDateTime.now());
41
42
43                  SensorHub.getInstance().publishReading(reading);
44
45                  double minSafe = PropertiesHelper.getMinThreshold(sensor.getSensorId(), sensor.getMinSafe());
46                  double maxSafe = PropertiesHelper.getMaxThreshold(sensor.getSensorId(), sensor.getMaxSafe());
47
48                  if (value < minSafe || value > maxSafe) {
49                      String msg = (value < minSafe) ? "Thap duoi nguong Min" : "Vuot nguong Max";
50                      Alert alert = new Alert(sensor.getSensorId(), sensor.getName(), value, msg);
51
52                      SensorHub.getInstance().publishAlert(alert);
53                  }
54              }
55          }
56      }
57  }
58
```

Sau khi phát Reading, engine lấy ngưỡng từ PropertiesHelper, dùng ngưỡng của Sensor làm giá trị dự phòng. Điều kiện cảnh báo là value < minSafe hoặc value > maxSafe; giá trị bằng một trong hai biên vẫn được xem là an toàn. SensorHub gọi từng listener theo thứ tự đăng ký.

GaugePanel lọc Reading theo sensorId, cập nhật currentValue, chọn màu xanh hoặc đỏ rồi gọi repaint(). LogTableModel thêm một hàng cho Reading và thêm một hàng khác khi có Alert. Vì vậy, một phép đo vượt ngưỡng có thể tạo hai hàng nhật ký, chứ không chỉ một hàng được đổi trạng thái.

3.5. Lưu cấu hình và giới hạn dữ liệu

PropertiesHelper nạp app.properties trong khối static, đọc và ghi UTF-8. Các khóa có dạng temp.min, temp.max, ph.min và ph.max. Khi lưu từ hộp thoại, hai cặp ngưỡng được ghi lần lượt; engine và gauge tra cứu bộ Properties trong bộ nhớ ở các lần cập nhật tiếp theo.

LogTableModel dùng LinkedList<Object[]> và MAX_ROWS = 200. Hàng mới thêm tại vị trí 0; khi kích thước vượt 200, hàng cuối bị xóa. Giới hạn này áp dụng cho tổng số hàng Reading và Alert, không phải 200 giây hoặc 200 phép đo cho từng cảm biến.

Các điểm cần bổ sung gồm kiểm tra NaN/vô cực trong setter, bắt lỗi số sai định dạng trong tệp cấu hình và phản hồi lỗi ghi tệp về hộp thoại. saveProperties() hiện chỉ in lỗi I/O, nên thông báo lưu thành công trên giao diện chưa bảo đảm dữ liệu đã ghi được xuống đĩa.

3.6. Biểu đồ và thiết bị chấp hành

Dự án đính kèm thư viện lib/jfreechart-1.5.6.jar. ChartWindow tạo JTabbedPane với hai tab: biểu đồ thời gian và biểu đồ cột cảnh báo. StandardChartTheme cấu hình font Segoe UI; việc hiển thị phụ thuộc font có trên máy chạy.

Biểu đồ thời gian có bốn TimeSeries theo các mã temp, ph, do, sal. setMaximumItemAge(60) giới hạn tuổi mẫu theo đơn vị Second; onReading() dùng addOrUpdate(new Second(), value). Dữ liệu bắt đầu được nhận khi cửa sổ đăng ký với Hub, không tự khôi phục dữ liệu trước lúc mở.

Biểu đồ cột dùng DefaultCategoryDataset và Map<String, Integer> để đếm mỗi sự kiện Alert. Mã nguồn khởi tạo nhãn DO nhưng khi xử lý tên "Oxy hòa tan (DO)" lại rút gọn thành "Oxy hòa tan". Sự không thống nhất này có thể tạo thêm một cột riêng; cần dùng sensorId làm khóa hoặc chuẩn hóa nhãn chung.

ChartWindow dùng DISPOSE_ON_CLOSE nên đóng cửa sổ con không kết thúc ứng dụng. Tuy nhiên, chưa có lệnh removeSensorListener() khi đóng; Hub vẫn có thể giữ tham chiếu đến cửa sổ. Việc mở và đóng nhiều lần cần được bổ sung bước hủy đăng ký.

Vẽ đồ thị thời gian thực Real-time:

```
99
100     @Override
101     public void onReading(Reading r) {
102
103         TimeSeries series = seriesMap.get(r.getSensorId());
104         if (series != null) {
105
106             series.addOrUpdate(new Second(), r.getValue());
107         }
108     }
109
```

Aerator cài đặt Controllable và SensorListener. Khi nhận Reading có mã do, lớp lấy ngưỡng tối thiểu từ PropertiesHelper, dự phòng bằng 4,0. DO thấp hơn ngưỡng làm bật thiết bị; DO bằng hoặc cao hơn ngưỡng làm tắt thiết bị.

```
39
40      @Override
41      public boolean isOn() {
42          return state;
43      }
44
45      @Override
46      public void onReading(Reading r) {
47
48          if (r.getSensorId().equals("do")) {
49              double minSafeDO = PropertiesHelper.getMinThreshold("do", 4.0);
50
51              if (r.getValue() < minSafeDO) {
52                  turnOn();
53              } else {
54
55                  turnOff();
56              }
57          }
58      }
```

turnOn() và turnOff() chỉ đổi biến state khi cần và in thông báo ra console. Chưa có lệnh gửi tới phần cứng, nút điều khiển thủ công hoặc hoạt ảnh quạt nước. pondId dùng trong thông báo; dữ liệu được lọc theo mã do chung nên cần bổ sung định danh ao khi mở rộng nhiều ao.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ GIAO DIỆN

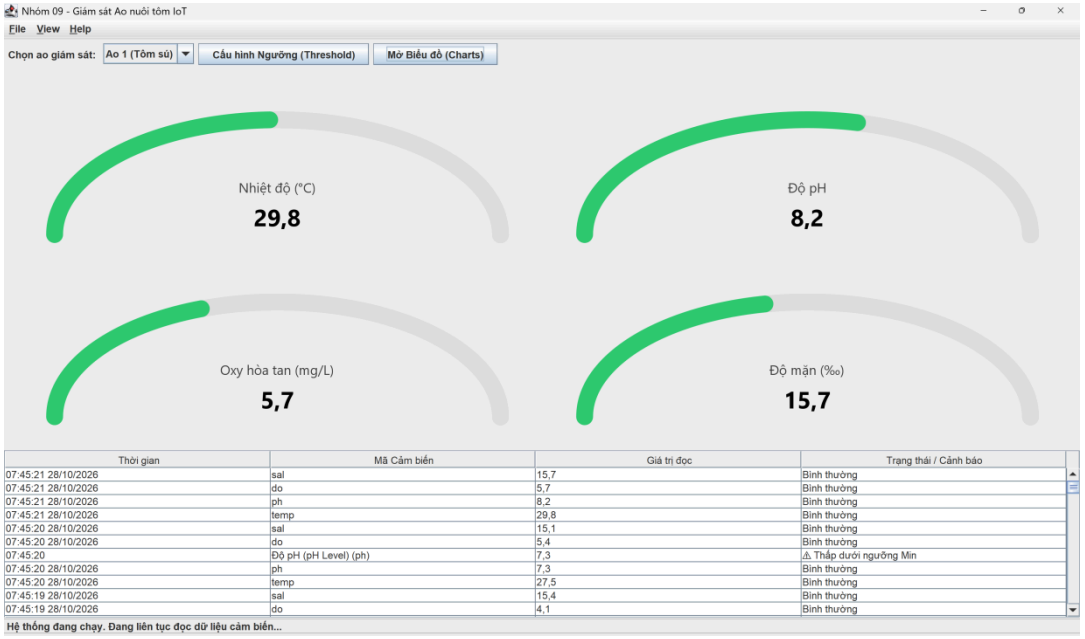
4.1. Cơ sở đánh giá và đối chiếu phiên bản

Bảng 4.1. Đối chiếu các điểm thay đổi giữa bản Word nền và mã nguồn được cập nhật.

Vị trí trong bản nền	Mô tả của bản nền	Đối chiếu mã nguồn mới
Mục 2.1 và 3.3	Hộp thoại cấu hình chỉ hỗ trợ nhiệt độ và pH.	ThresholdDialog hiện có 8 JSpinner để chỉnh Min/Max của cả bốn cảm biến temp, ph, do và sal.
Mục 3.2	Độ mặn mô phỏng với hệ số Gaussian 0,8.	SalinitySensor.read() đang dùng nextGaussian() × 4,5; cần dùng giá trị 4,5 khi trình bày kết quả của mã nguồn mới.
Mục 3.3	Constructor của Sensor chưa kiểm tra ngưỡng.	Constructor hiện ném IllegalArgumentException khi minSafe ≥ maxSafe; setter vẫn ném InvalidThresholdException.
Mục 3.5	Properties chỉ ghi temp và ph.	Lớp dialog hiện lần lượt ghi cả temp, ph, do, sal vào app.properties.
Mục 3.6	BarChart có khả năng tạo thêm cột DO khác tên.	Mã mới khởi tạo “Oxy hòa tan”, trùng tên rút gọn khi cập nhật. Kiểm thử T16 ghi nhận bốn cột; hình ảnh gửi trước vẫn có năm cột.
Mục 3.7	Ghi nhận chưa có giao diện cấu hình DO và độ mặn.	Đã có giao diện mới; những hạn chế khác như status bar, mức vàng và CSV vẫn còn.

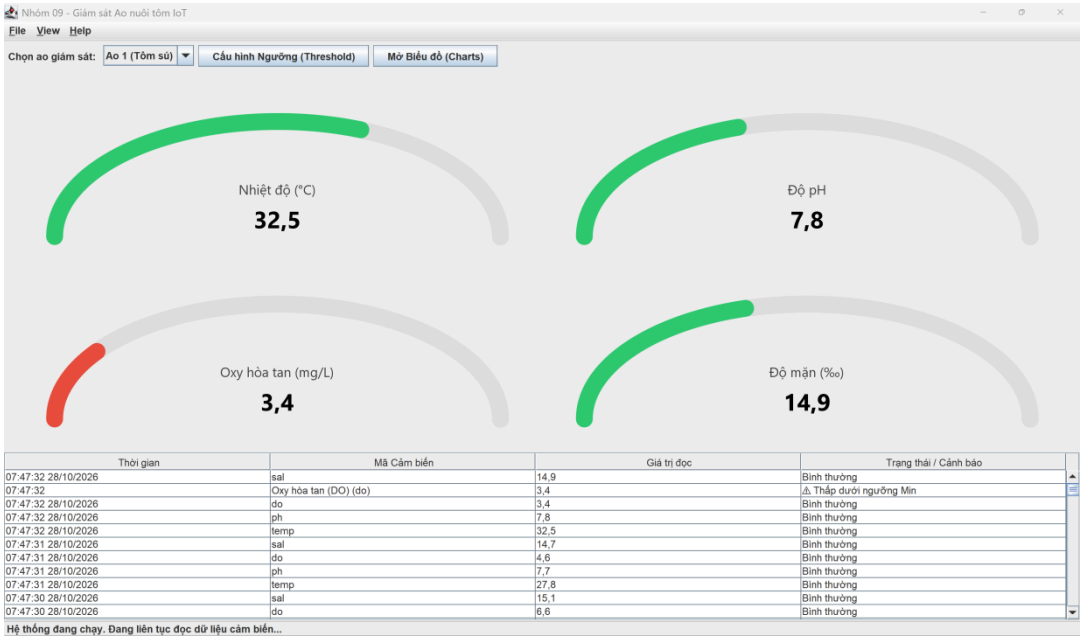
4.2. Cửa sổ chính, dashboard và nhật ký

MainWindow kế thừa BaseWindow, sử dụng BorderLayout; DashboardPanel bố trí bốn GaugePanel theo GridLayout(2,2), tương ứng nhiệt độ, pH, oxy hòa tan (DO) và độ mặn. Dữ liệu mẫu khởi tạo một trang trại với một ao P1 (Ao 1 – Tôm sú, diện tích 500 m²). Người quản lý quan sát các chỉ số bằng cung tròn do Graphics2D vẽ, đồng thời theo dõi JTable ở phần dưới. Mỗi nhịp Timer 1.000 ms, SimulationEngine lấy một Reading cho từng Sensor, phát qua SensorHub và có thể phát thêm Alert nếu vượt ngưỡng.



Hình 4.1. Toàn cảnh dashboard ở trạng thái các chỉ số nằm trong vùng an toàn (ảnh do nhóm cung cấp).

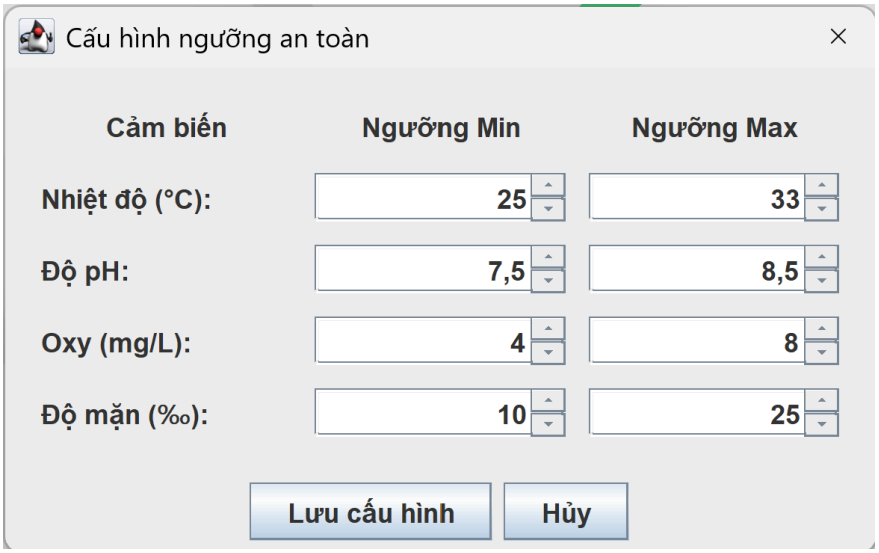
Tại trạng thái bình thường, cung đo hiển thị màu xanh. Khi oxy hòa tan giảm dưới ngưỡng, GaugePanel của DO chuyển đỏ và LogTableModel ghi thêm bản tin cảnh báo. Một Reading bị vượt ngưỡng có thể sinh hai dòng nhật ký: dòng Reading mang trạng thái "Bình thường" và dòng Alert bổ sung; đó là cơ chế ghi hai sự kiện, không phải bộ đếm sai. Bộ đệm dữ liệu tối đa 200 dòng tính chung Reading và Alert, các hàng cũ nhất được loại bỏ khi vượt giới hạn.



Hình 4.2. Dashboard khi DO ở dưới ngưỡng: cung đo chuyển đỏ và bảng nhật ký bổ sung cảnh báo

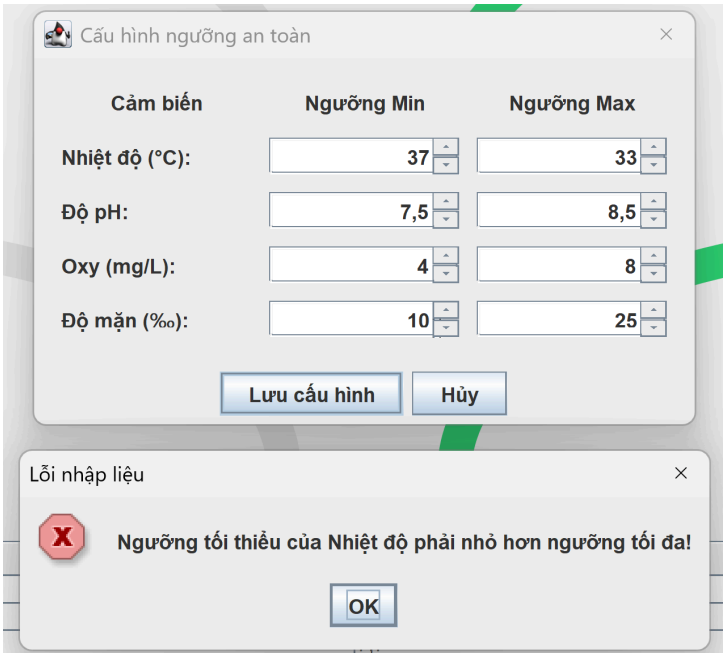
4.3. Hộp thoại cấu hình ngưỡng và xử lý dữ liệu sai

ThresholdDialog extends JDialog theo chế độ modal. Bốn cặp ngưỡng được nhập bằng JSpinner: nhiệt độ (0–50 °C, bước 0,5), pH (0–14, bước 0,1), DO (0–10 mg/L, bước 0,1) và độ mặn (0–35 ‰, bước 0,5). Khi nhấn Lưu cấu hình, saveSettings() kiểm tra mọi cặp Min < Max; nếu hợp lệ, gọi PropertiesHelper.setThreshold() lần lượt cho bốn mã cảm biến. Lần đọc tiếp theo trong SimulationEngine và GaugePanel sử dụng ngưỡng mới từ Properties.



Hình 4.3. Hộp thoại cấu hình Min/Max cho đủ bốn cảm biến trong mã nguồn mới.

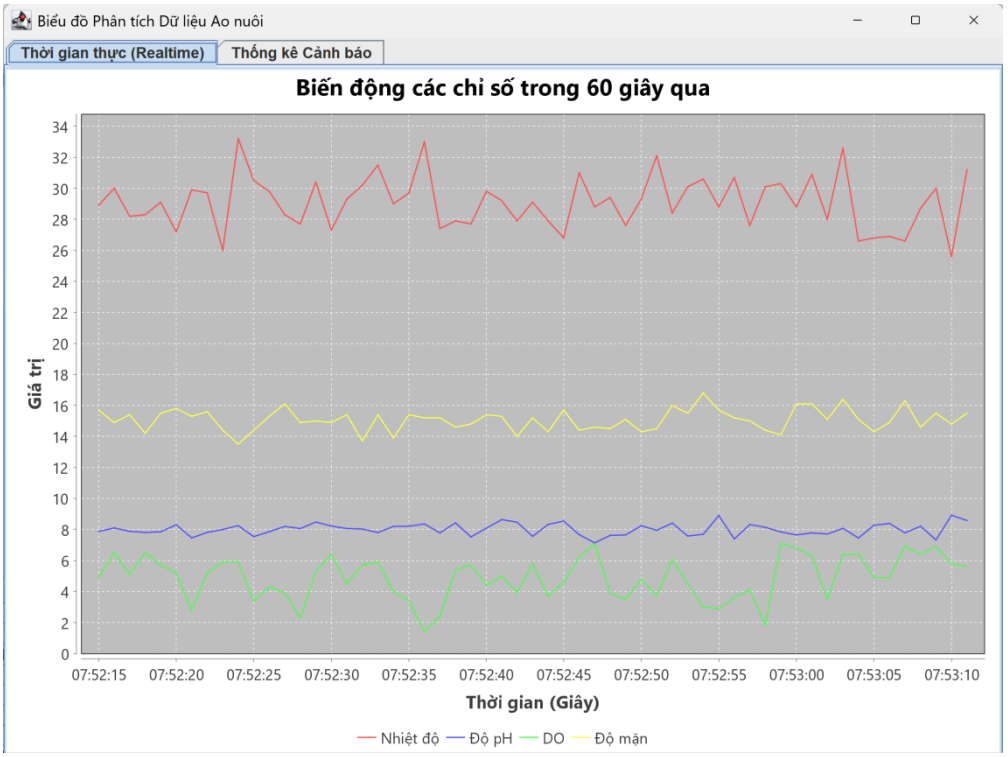
Khi Min không nhỏ hơn Max, mã xử lý phát sinh InvalidThresholdException và chuyển thông điệp sang JOptionPane, thay vì đóng ứng dụng. Hình 4.4 minh họa trường hợp người dùng nhập Min nhiệt độ lớn hơn Max. Cần phân biệt rằng kiểm tra trong hộp thoại được viết trực tiếp ở saveSettings(), chưa gọi Sensor.setSafeThresholds(); vì vậy đây là hai đường kiểm tra cùng quy tắc nhưng chưa được kết nối vào một luồng cập nhật duy nhất.



Hình 4.4. Thông báo lỗi khi nhập ngưỡng nhiệt độ không hợp lệ; hệ thống vẫn giữ hộp thoại để sửa.

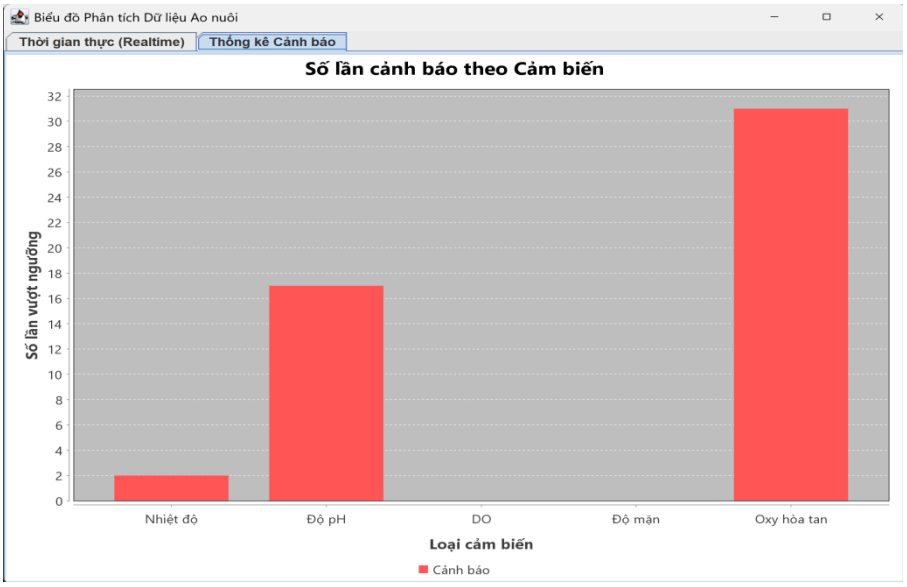
4.4. Hai biểu đồ JFreeChart

ChartWindow extends JFrame và cài đặt SensorListener; cửa sổ có hai thẻ: “Thời gian thực (Realtime)” và “Thống kê Cảnh báo”. Biểu đồ thời gian dùng TimeSeriesCollection với bốn series theo mã temp, ph, do, sal; dữ liệu được thêm bằng addOrUpdate(new Second(), value). Hàm setMaximumItemAge(60) giới hạn tuổi mẫu theo đơn vị giây. ChartWindow chỉ nhận sự kiện từ lúc được mở nên biểu đồ không tự có dữ liệu lịch sử trước đó.



Hình 4.5. TimeSeriesChart trực quan hóa các chỉ số cảm biến trong khoảng thời gian gần nhất.

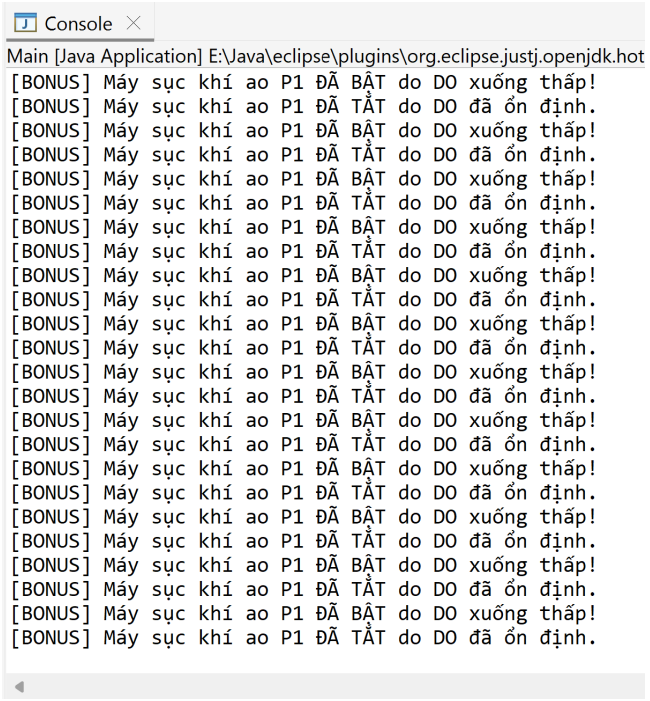
Biểu đồ cột sử dụng DefaultCategoryDataset và Map<String,Integer> để đếm số lần Alert.



Hình 4.6. Ảnh BarChart do nhóm cung cấp.

`

Ở mức BONUS, Aerator cài đặt Controllable và SensorListener. Khi nhận Reading mã do với giá trị thấp hơn ngưỡng tối thiểu, thiết bị giả lập chuyển state = true và in thông báo bật; khi chỉ số phục hồi, state chuyển về false và in thông báo tắt. Ảnh console ghi các lần chuyển trạng thái; hệ thống chưa gửi lệnh tới phần cứng thật. Về nghiệp vụ, điều kiện này chỉ tự xử lý DO thấp, không bật máy theo trường hợp DO vượt Max.



Hình 4.8. Nhật ký console về việc máy sục khí giả lập tự bật và tắt theo ngưỡng DO.

4.6. Tổng hợp mức độ đáp ứng thẻ Đề 9

Bảng 4.2. Đánh giá thực trạng tám bước triển khai theo thẻ đề.

Bước	Yêu cầu trọng tâm	Đánh giá đối chiếu mã nguồn mới
1	Sensor trừu tượng; bốn lớp con và Gaussian.	Có đủ bốn lớp con, override read(); constructor Sensor chặn Min ≥ Max. Đầu vào NaN/vô cực chưa được ràng buộc.
2	Farm, Pond, Reading bất biến.	Có đủ mô hình; Reading final/private final; chạy mẫu một ao P1.
3	Observer: SensorHub/SensorListener.	Có đăng ký, phát và hủy đăng ký; ChartWindow chưa tự hủy listener khi đóng.
4	MainWindow, chọn ao, gauge xanh–vàng–đỏ.	Có bố cục và bốn gauge; chưa có mức vàng và chưa xử lý đổi ao thực sự.
5	Timer 1 giây, JTable tối đa 200, status bar đỏ.	Có Timer/nhật ký/gauge đỏ; status bar chưa đổi đỏ theo Alert.
6	ThresholdDialog cho từng cảm biến, Properties.	Đã chỉnh Min/Max cho cả bốn cảm biến; lỗi ghi tệp chưa phản hồi lên giao diện.
7	TimeSeriesChart và BarChart.	Hai loại biểu đồ có dữ liệu động; mã mới không tách đôi cột DO; còn vấn đề đơn vị chung và giải phóng listener.
BONUS	Aerator tự động; xuất CSV.	Aerator bật/tắt theo DO và console; chưa có chức năng xuất CSV.

CHƯƠNG 5. KIỂM THỬ HỆ THỐNG

5.1. Mục tiêu, môi trường và cách tiến hành

Nhóm mã nguồn được kiểm tra theo hai lớp bằng chứng. Thứ nhất, rà soát trực tiếp lớp, phương thức, luồng sự kiện, lỗi ngoại lệ và các tệp cấu hình. Thứ hai, biên dịch và chạy bộ test độc lập, JFreeChart 1.5.6 đính kèm và môi trường hiển thị ảo Xvfb cho các kiểm thử Swing. Kết quả do chương trình kiểm thử in ra; đây không phải kết quả thử nghiệm trên cảm biến hay ao tôm thực.

5.2. Bảng test case và kết quả thực tế

Bảng 5.1. Bộ 17 trường hợp kiểm thử độc lập đối với mã nguồn mới.

Mã	Mô tả	Dữ liệu vào / thao tác	Kết quả mong đợi	Kết quả thực tế	Kết luận
T01	Lấy SensorHub hai lần	getInstance() × 2	Cùng một thể hiện	Cùng instance	Đạt
T02	Ngưỡng khởi tạo sai	temp, Min=33; Max=25	Bị từ chối	IllegalArgumentException	Đạt
T03	Cập nhật ngưỡng hợp lệ	Min=28; Max=30	Cập nhật hai giới hạn	Đã cập nhật	Đạt
T04	Cập nhật Min ≥ Max	Min=31; Max=30	Ném lỗi, giữ giá trị cũ	InvalidThresholdException, không đổi	Đạt
T05	Giá trị ở biên	28; 30; 27,9	Biên an toàn; dưới Min cảnh báo	Đúng cả ba trường hợp	Đạt
T06	Kiểm tra Reading	Reflection: final, setter	Dữ liệu bất biến	Lớp và trường final, không setter	Đạt
T07	Sửa danh sách Sensor	pond.getSensors() .clear()	Không thể sửa từ bên ngoài	UnsupportedOperation Exception	Đạt
T08	Sửa danh sách Pond	farm.getPonds().clear()	Không thể sửa từ bên ngoài	UnsupportedOperation Exception	Đạt
T09	Observer đăng ký lặp	Đăng ký 2 lần, phát 2 loại event	Nhận mỗi event đúng 1 lần	1 Reading + 1 Alert	Đạt
T10	Giới hạn nhật ký	Phát 250 Reading	Tối đa 200 dòng	200 dòng	Đạt
T11	Giá trị không âm	1000 lần read() DO và độ mặn	Không xuất hiện giá trị âm	Không phát hiện giá trị âm	Đạt

Mã	Mô tả	Dữ liệu vào / thao tác	Kết quả mong đợi	Kết quả thực tế	Kết luận
T12	Đọc ngưỡng bốn cảm biến	Ghi temp/ph/do/sal	Trả về ngưỡng đã ghi	Đọc được do.max=8, sal.min=10	Đạt
T13	Bật/tắt Aerator	DO=3,1 rồi 5,0	Bật sau thấp; tắt khi phục hồi	state true rồi false	Đạt
T14	Cấu hình đủ 4 sensor	Đếm JSpinner trong dialog	8 spinner	Có 8 spinner	Đạt
T15	Hai biểu đồ	Đếm tab ChartWindow	Có 2 tab	Có 2 tab	Đạt
T16	Đồng nhất nhãn DO	Phát Alert "Oxy hòa tan (DO)"	Dataset vẫn 4 cột	4 cột	Đạt
T17	Giải phóng observer	Tạo rồi dispose ChartWindow	Số listener trở lại ban đầu	Listener vẫn còn sau dispose	Chưa đạt

Kết quả tổng hợp: 16/17 ca kiểm thử đạt; 1/17 chưa đạt. Tỉ lệ này chỉ phản ánh các phép thử có trong bộ kiểm tra, không đại diện cho tỉ lệ hoàn thành toàn bộ đề bài. Riêng T11 là kiểm thử ngẫu nhiên hữu hạn (1.000 lần), không phải chứng minh toán học cho mọi giá trị có thể phát sinh.

5.3. Phân tích trường hợp chưa đạt và các giới hạn cần kiểm tra tiếp

T17 – Vòng đời listener: ChartWindow gọi addSensorListener(this) trong constructor nhưng không cài đặt hủy đăng ký khi dispose(). Do SensorHub Singleton còn giữ tham chiếu, đóng nhiều cửa sổ liên tiếp có thể làm tăng số đối tượng vẫn nhận sự kiện, gây tiêu hao tài nguyên hoặc lưu giữ dữ liệu không cần thiết. Hướng sửa: thêm WindowListener/WindowAdapter cho windowClosed() để gọi SensorHub.getInstance().removeSensorListener(this).

Các điểm chưa đạt yêu cầu theo đối chiếu tĩnh, chưa được đưa vào 17 ca tự động nói trên: (i) toolbar chỉ có khung nhưng chưa bổ sung JButton có icon; (ii) status bar không đổi màu đỏ khi nhận Alert; (iii) GaugePanel chỉ đổi xanh/đỏ, thiếu vàng; (iv) JComboBox chọn ao chưa có listener để chuyển dữ liệu; (v) chưa có xuất CSV; (vi) MainWindow và AboutDialog còn ghi "Nhóm 09" thay vì "Nhóm 06". Đây là những đầu việc thực tế để nhóm kiểm tra trước bảo vệ.

5.4. Kịch bản demo và kiểm chứng mở rộng

Bảng 5.2. Kịch bản đề xuất để nhóm thực hiện trong buổi bảo vệ.

Bước	Thao tác đề xuất	Bằng chứng quan sát
1	Khởi chạy Main, chọn ao P1.	Bốn gauge cập nhật theo Timer và nhật ký có dòng mới.
2	Đợi DO giảm dưới 4 mg/L hoặc điều chỉnh ngưỡng thích hợp.	Gauge DO đỏ; xuất hiện Alert; console có thông báo bật sục khí.
3	Mở ThresholdDialog, thử Min nhiệt độ ≥ Max.	JOptionPane báo lỗi; hộp thoại vẫn có thể sửa.
4	Lưu ngưỡng hợp lệ cho bốn cảm biến.	Tệp app.properties cập nhật; ngưỡng mới tác động vào lần đo sau.
5	Mở TimeSeriesChart và BarChart.	Có hai tab; chuỗi tăng theo nhịp, cột tăng khi có Alert.
6	Đóng và mở biểu đồ nhiều lần.	Đếm listener và xác minh không tích lũy sau khi áp dụng bản sửa.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Đề tài Smart Aquaculture Monitor đã xây dựng được mô hình desktop Java có tách biệt lớp dữ liệu cảm biến, điều phối lấy mẫu, trung tâm phát sự kiện và giao diện hiển thị. Bốn loại Sensor hiện thực read() theo cách riêng, thể hiện kế thừa và đa hình; Sensor, Reading và các danh sách chỉ đọc minh họa đóng gói; SensorListener/Controllable và BaseWindow minh họa trừu tượng. Bộ Timer, dashboard, nhật ký 200 dòng, hai biểu đồ JFreeChart, hộp thoại cấu hình đủ bốn cảm biến và mô hình Aerator tạo nên một luồng giám sát có thể trình diễn.

6.2. Hạn chế của hệ thống

• Dữ liệu hiện dựa trên mô phỏng phân phối chuẩn Gauss, chưa kết nối trực tiếp với phần cứng vi điều khiển Arduino/ESP32 thực tế.

• Chưa tích hợp hệ thống phân quyền (người xem và kỹ sư quản trị).

6.3. Khó khăn và bài học kinh nghiệm

• Lỗi kỹ thuật quan trọng: Constructor của BaseWindow gọi initUI() trước khi constructor của MainWindow chạy xong. Nhóm đã rút ra bài học bắt buộc phải khởi tạo các biến bên trong initUI() thay vì khởi tạo tại chỗ khai báo biến.

• Bài học quản lý nhóm: Áp dụng phân chia nhánh Git (feature branching) và quy ước đặt tên commit (feat:, fix:, docs:) giúp 10 thành viên phối hợp hiệu quả, không bị xung đột code.

6.4. Hướng phát triển trong tương lai

• Kết nối giao thức MQTT / WebSocket để tiếp nhận dữ liệu từ cảm biến thực tế ngoài đầm tôm.

• Tích hợp cơ sở dữ liệu quan hệ MySQL và xây dựng thuật toán Machine Learning dự báo chất lượng nước.

TÀI LIỆU THAM KHẢO

1. *Học viện Công nghệ Bưu chính Viễn thông. (2026). Phát triển ứng dụng Java — Bài tập lớn Lập trình hướng đối tượng: Thẻ đề số 9, hướng dẫn báo cáo và tiêu chí đánh giá. Tài liệu học phần INT1332.*
2. *Nhóm 06. (2026). Smart Aquaculture Monitor: mã nguồn Java trên https://github.com/0fbiscuit/OOP_Nhom06_SmartAquacultureMonitor*
3. *JFreeChart Project, JFreeChart 1.5 Developer Guide and API Specifications, [Online]. Available: https://www.jfree.org/jfreechart/.*
4. *Oracle. Java Platform, Standard Edition 17 – API Specification. https://docs.oracle.com/en/java/javase/17/docs/api/*

5. *Oracle. The Java Tutorials – Performing Custom Painting.* https://docs.oracle.com/javase/tutorial/uiswing/painting/index.html
6. *Object Management Group (OMG). (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1.* https://www.omg.org/spec/UML/2.5.1
7. *Oracle. Java SE 17 – Class Random (java.util.Random).* https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html

PHỤ LỤC A. NHẬT KÝ HỖ TRỢ BẰNG AI:

Nhóm có sử dụng AI nhưng không vì thế mà phụ thuộc vào AI. Nhóm đã có những đánh giá, triển khai. Cũng như xem xét lại code, cũng như các phần khác. Có chủ động sửa, và viết lại code.

Gemini được dùng để tạo bảng phân công công việc cho từng thành viên, cũng như tạo code mẫu cho từng thành viên.

ChatGPT được sử dụng để đọc và đối chiếu tài liệu giao đề, phân tích mã nguồn do nhóm cung cấp, gợi ý bố cục Chương 4–6 và soạn bộ kiểm thử độc lập. Phép thử được thực thi trên bản sao độc lập của mã nguồn. Nhóm cần kiểm tra lại toàn bộ nhận xét trước khi ký báo cáo và bổ sung trung thực những công cụ AI đã dùng trong quá trình lập trình trước đó (nếu có).

Bảng A.1. Hoạt động AI hỗ trợ trong lần rà soát và biên tập báo cáo.

Hoạt động	Mục đích / dữ liệu đầu vào	Kiểm chứng của người phụ trách
Đọc yêu cầu học phần	Sử dụng tài liệu giao bài tập lớn của giảng viên để hỗ trợ phân tích yêu cầu, xác định các chức năng bắt buộc, tiêu chí đánh giá và nội dung cần trình bày trong báo cáo.	Các thành viên đối chiếu kết quả phân tích với đề bài gốc, kiểm tra mức độ đáp ứng từng yêu cầu và thống nhất nội dung cần thực hiện.
Rà soát code	Sử dụng mã nguồn Java của dự án để hỗ trợ kiểm tra cấu trúc chương trình, cách tổ chức các lớp, việc áp dụng nguyên lý OOP và sự liên kết giữa các chức năng.	Nhóm kiểm tra các lớp và phương thức liên quan, biên dịch chương trình, đối chiếu chức năng với mã nguồn và xác định những phần cần điều chỉnh.
Kiểm thử độc lập	Sử dụng mã nguồn dự án để hỗ trợ xây dựng các trường hợp kiểm thử, kiểm tra xử lý dữ liệu, cơ chế phát sự kiện, ngoại lệ và hoạt động của các thành phần giao diện.	Chạy 17 ca test, ghi 16 đạt/1 chưa đạt; giữ log đi kèm. Đối chiếu kết quả với hành vi mong đợi và lưu lại minh chứng kiểm thử.

Hoạt động	Mục đích / dữ liệu đầu vào	Kiểm chứng của người phụ trách
Biên soạn báo cáo	Sử dụng tài liệu yêu cầu của giảng viên, mã nguồn Java, sơ đồ UML và hình ảnh giao diện để hỗ trợ tổng hợp, sắp xếp nội dung và hoàn thiện báo cáo.	Các thành viên rà soát nội dung được hỗ trợ, kiểm tra tính chính xác của các đoạn mã, sơ đồ và hình ảnh; chỉnh sửa cách diễn đạt, bổ sung phân tích và đối chiếu với chức năng thực tế của chương trình.

PHỤ LỤC B. LỊCH SỬ GIT VÀ THÔNG TIN CẦN HOÀN THIỆN

Đường dẫn lưu trữ : https://github.com/0fbiscuit/OOP_Nhom06_SmartAquacultureMonitor/

Biểu đồ thống kê đóng góp mã nguồn (Contributors) của các thành viên trong nhóm trên GitHub

